

Creating and Manipulating Databases in Python



**** Note:** The code chunks below should be run in the following order ******

Using Databases With Python

We have shown how to create, update and query a database using DB Browser for SQLite and in R. Now we will illustrate how the same thing can be done in Python. Again we will continue using the university example.

Connecting to Databases Using Python

We import the module **sqlite3** and use the function `connect()` to create an object, `conn`, to connect to the SQLite driver to manipulate the database `University.db`. If the database `University.db` exists in your working directory, the following code chunk will remove it.

```
# This makes sure you can run this notebook multiple times without errors
import os
try:
    os.remove('University.db')
except OSError:
    pass
import sqlite3
conn = sqlite3.connect('University.db')
```

Creating Tables Using Python

Now we are going to create some tables to the database `University.db`. Like before, we will create the tables using the data saved in the CSV files. We first load the CSV files into `DataFrame` in Python:

```
import pandas as pd

student = pd.read_csv("student.csv")
course = pd.read_csv("course.csv")
grade = pd.read_csv("grade.csv")
```

We then write record stored in `DataFrames` `student`, `grade` and `course` as tables to the database `University.db` using the `DataFrame` method `to_sql()`.

```
# index = False to ensure the DataFrame row index is not written into the SQL table
s

student.to_sql('Student', con = conn, index = False)

course.to_sql('Course', con = conn, index = False)

grade.to_sql('Grade', con = conn, index = False)
```

Again, we can check if the database is created properly by opening the database in DB Browser for SQLite and browse the tables.

Manipulate Databases Using Python

We can manipulate databases in Python by the `execute()` and `fetchall()` methods from the **sqlite3** module which performs SQL commands. This allows us to leverage the SQL commands we have learned to manipulate the databases in Python. We first need to create a cursor object `c`:

```
c = conn.cursor()
```

After that we can execute the SQL commands we learned before using the function `execute()` and `fetchall()`. For example, if we want to get all the tables in the database, we can run:

```
c.execute('''
SELECT name
FROM sqlite_master
WHERE type='table'
''')

<sqlite3.Cursor object at 0x1447c1f80>
```

The result is not returned until we run `fetchall()`:

```
c.fetchall()

[('Student',), ('Course',), ('Grade',)]
```

We can see there are three tables in the database. If we want to browse the table `Student` we can run (here we display the results as `pandas DataFrame`):

```
import pandas as pd

q = c.execute("SELECT * FROM Student").fetchall()

pd.DataFrame(q)

   0          1  2
0  201921323    Ava Smith  2
1  201832220    Ben Johnson  3
2  202003219  Charlie Jones  1
3  202045234    Dan Norris  1
```

```
4  201985603      Emily Wood  1
5  201933222    Freddie Harris  2
6  201875940      Grace Clarke  2
```

Note here we combine the use of `execute()` and `fetchall()` in one line.

Add a New Table

We can add a new table by running the SQL command through `execute()`:

```
c.execute('''
CREATE TABLE Teacher (staff_id TEXT PRIMARY KEY,
name TEXT)
''')
<sqlite3.Cursor object at 0x1447c1f80>
conn.commit() # save (commit) the changes
```

When we list the tables, we can see four tables.

```
c.execute('''
SELECT name
FROM sqlite_master
WHERE type='table'
''').fetchall()
[('Student',), ('Course',), ('Grade',), ('Teacher',)]
```

Delete a Table

We can delete a table by running the SQL command through `execute()`:

```
c.execute("DROP TABLE Teacher")
<sqlite3.Cursor object at 0x1447c1f80>
conn.commit()
```

When we list the tables, we can see three tables.

```
c.execute('''
SELECT name
FROM sqlite_master
WHERE type='table'
''').fetchall()
[('Student',), ('Course',), ('Grade',)]
```

Insert Tuples/Rows

Insert the year 1 student Harper Taylor with student ID 202029744 to Student:

```
c.execute("INSERT INTO Student VALUES(202029744, 'Harper Taylor', 1)")
<sqlite3.Cursor object at 0x1447c1f80>
conn.commit()
```

When we browse the table, we can see the new row is added.

```
q = c.execute("SELECT * FROM Student").fetchall()
pd.DataFrame(q)
```

	0	1	2
0	201921323	Ava Smith	2
1	201832220	Ben Johnson	3
2	202003219	Charlie Jones	1
3	202045234	Dan Norris	1
4	201985603	Emily Wood	1
5	201933222	Freddie Harris	2
6	201875940	Grace Clarke	2
7	202029744	Harper Taylor	1

Update Tuples/Rows

Update the student ID of student Harper Taylor to 201929744:

```
c.execute('''
UPDATE Student
SET student_id = "201929744"
WHERE name = "Harper Taylor"
''')
<sqlite3.Cursor object at 0x1447c1f80>
conn.commit()
```

When we browse the table, we can see the row has changed.

```
q = c.execute("SELECT * FROM Student").fetchall()
pd.DataFrame(q)
```

	0	1	2
0	201921323	Ava Smith	2
1	201832220	Ben Johnson	3
2	202003219	Charlie Jones	1
3	202045234	Dan Norris	1
4	201985603	Emily Wood	1
5	201933222	Freddie Harris	2

```
6 201875940    Grace Clarke  2
7 201929744    Harper Taylor 1
```

Delete Tuples/Rows

Delete the record for the student Harper Taylor from table `Student`:

```
c.execute('''
DELETE FROM Student
where name = "Harper Taylor"
''')
<sqlite3.Cursor object at 0x1447c1f80>
conn.commit()
```

When we browse the table, we can see the row has been removed.

```
q = c.execute("SELECT * FROM Student").fetchall()
pd.DataFrame(q)
```

	0	1	2
0	201921323	Ava Smith	2
1	201832220	Ben Johnson	3
2	202003219	Charlie Jones	1
3	202045234	Dan Norris	1
4	201985603	Emily Wood	1
5	201933222	Freddie Harris	2
6	201875940	Grace Clarke	2

Disconnecting From the Database

After we finish manipulating the database, we can close the connection using the method `close()` on `conn`:

```
conn.close()
```

Useful Links and Resources

- [sqlite3](#)

Querying Databases in Python

**** Note:** The code chunks below should be run in the following order ******

Using Databases With Python

In the first notebook we have seen how to use Python to connect, create and manipulate a database. Now we will use Python to query databases using the database `University.db` that we have created in the first notebook.

Connecting to Databases Using Python

We first connect to the database `University.db` and list all the tables. We should see all the three tables `Student`, `Course`, and `Grade` that we have created in the first notebook.

```
import sqlite3

conn = sqlite3.connect('University.db')

c = conn.cursor()

c.execute("SELECT name FROM sqlite_master WHERE type='table'").fetchall()

[('Student',), ('Course',), ('Grade',)]
```

Querying Databases Using Python

We can query databases in Python by the `execute()` and `fetchall()` methods from the `sqlite3` module which performs SQL commands. Here we display the results as a `Pandas` data frame. The SQL commands used here have been discussed in the previous notebooks.

Example 1: Get Grades of the Course `course_id ST101`

```
q1 = c.execute('''
SELECT final_mark
FROM Grade
WHERE course_id = 'ST101'
''').fetchall()

import pandas as pd
pd.DataFrame(q1)

0
0 78
1 60
```

Example 2: Get Names of Students in Alphabetical Order

```
q2 = c.execute('''
SELECT Student.name
FROM Grade, Student
WHERE Grade.course_id = 'ST101' AND Student.student_id = Grade.student_id
ORDER BY Student.name
''').fetchall()

pd.DataFrame(q2)

```

	0
0	Ava Smith
1	Charlie Jones
2	Emily Wood

Example 3: Get Courses Taken by Ava Smith or Freddie Harris

```
q3 = c.execute('''
SELECT DISTINCT Course.name
FROM Student, Grade, Course
WHERE (Student.name = 'Ava Smith' OR Student.name = 'Freddie Harris') AND
Student.student_id = Grade.student_id AND Course.course_id = Grade.course_id
''').fetchall()

pd.DataFrame(q3)

```

	0
0	programming for data science
1	Managing and Visualising Data
2	Databases

Example 4: Calculate Average Mark for Each Course

```
q4 = c.execute('''
SELECT course_id, AVG(final_mark) as avg_mark
FROM Grade
GROUP BY course_id
''').fetchall()

pd.DataFrame(q4)

```

```
''').fetchall()

pd.DataFrame(q4)

   0      1
0  ST101  61.666667
1  ST115  82.333333
2  ST207  66.500000
```

Disconnecting From Databases

After we finish manipulating the database, we can close the connection using the method `close` on `conn`:

```
conn.close()
```

Useful Links and Resources

- `sqlite3`